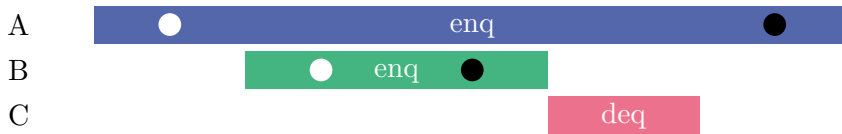


9.1

```
public void enq(T x) {  
    int i = tail.getAndIncrement(); // E.1  
    items[i].set(x);                // E.2  
}
```

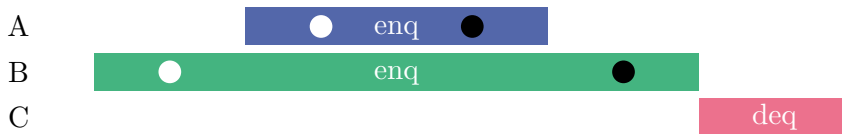
An example execution showing that the linearization point for `enq` cannot occur at line E.1 (white points are E.1s, black points are E.2s):



In the example above, `deq` will return the element inserted by thread B, but thread A has completed E.1 earlier, and if we consider it to be the linearization point, we should've got the element it tried to insert.

The “inflation” of the first `enq` might be due to thread A working slower than its siblings.

An example execution showing that the linearization point for `enq` cannot occur at line E.2:



In this example, even though A completed E.2 earlier than B did, B still has put its element further to the left of the array because it finished E.1 before A did. Thus `deq`, going from left to right, will encounter B's element first. If we consider E.2 to be a linearization point, this should not be a valid execution.

We have shown that neither E.1 nor E.2 are linearization points for `enq`, yet that does not mean that `enq` is not linearizable. Definition of linearizability does not state the linearization point is fixed, it depends on the outcome and the order of operations.